

Workshop Case Study

Trainer's application: Enterprise IT Service Desk

An organization has employees who raise IT service requests.

Example:

Employee → raises laptop/access/network/software issue → Service Desk → assigns technician → technician works on it → resolves → employee confirms → ticket closed.

Main actors

Role Responsibilities

Employee Create and view own tickets

Support Engineer View assigned tickets, update status

Team Lead Assign/reassign tickets, monitor team

Admin Manage users/categories/configuration

Core entities

User

Ticket

Category

Comment

Attachment

AuditLog

Ticket lifecycle

NEW → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED

With Possible:

IN_PROGRESS → ON_HOLD

ON_HOLD → IN_PROGRESS

This gives us genuine business logic rather than just:

POST /students

GET /students

PUT /students

DELETE /students

2. Student Teams Build Different Applications

This is important.

Don't ask every team to build the same Helpdesk application.

Trainer builds:

IT Service Desk

Teams choose one of these:

Option A — College Service Request System

Students/Faculty raise requests for:

Bonafide certificate

ID card

Hostel

Transport

Library

IT support

Option B — Hospital Support Request System

Departments raise:

Equipment issue

Maintenance

IT issue

Facility request

Option C — Corporate Facility Management

Employees raise:

AC issue

Electrical issue

Housekeeping

Meeting room

Security

IT

Option D — HR Employee Service Portal

Requests such as:

Leave clarification

Payroll query

Experience letter

Asset request

Onboarding request

Option E — E-commerce Customer Support

Customer:

Order problem → Support Ticket → Agent → Resolution → Closure

The domain changes, but the engineering architecture remains almost identical.

That is excellent for assessment because we can see whether students actually understand the architecture rather than copying the trainer.

Day Wise Schedule:

Day1:

Git, Github and commands

MongoDB commands

Mongo Cloud

Account Creation, setting up and connectivity. Git Desktop, Node, Python, Postman, VSC installation. Python topics such as REST API consumption. Test APIs using Postman. Docker to run local mongo. Python to connect to mongo local and remote.

Day2 and Day3

FastAPI, Postman and Backend App Completion. Docker to run backend app.

Day4 and Day5

Frontend app development using ReactJS where they will learn required concepts from JS, Bootstrap. Docker and docker compose to run backend and frontend orchestration.

Day6

A simple AI integration for our App (Like Data Analytics API or Chatbot or some plugin which students can learn and excites them)

1. Dockers will be used.
2. One App trainer will implement and another Students will implement on their own.
3. Every day 10 MCQ assessment will be given
4. Last day 20 MCQs assessment that covers all the topics.

Our major focus is on REST API, Fast API, 30% on Frontend 10% on database. Last day we will demonstrate how to use dockers and integration with gen AI.

Python, Node JS, Git Desktop, Visual Studio Code, Docker Desktop, Thunder Client extension on VSC / Postman. All are downloadable and latest version.

High speed internet to get connected to cloud mongo db.

Our team will create Google drive doc to have list of download pages. Will share it during the training.

APP INSTALLATIONS:

Python, Node JS, Git Desktop, Visual Studio Code, Docker Desktop, Thunder Client Extension on VS Code, Postman. All the Apps must be of latest version.

High speed internet to get connected to cloud MongoDB is necessary during training.